***Taller de base de datos***

*Evaluación Parcial N°2 · BDY1103 · Implementación de programas PL/SQL*

***30/10/2026***

***Profesor***

- ***Cristian Gandolfi***

***Integrantes y responsabilidades***

- ***Diego Jaramillo***
- ***Italo Campodonico***
- ***Jose Salas***

# 1. Introducción

## 1.1. Contexto del negocio

La agencia de scouting y representación deportiva (Football Talent Academy Agency) administra el seguimiento de futbolistas profesionales, sus evaluaciones técnicas, pruebas físicas, estado médico y las negociaciones de transferencia con clubes compradores. El proceso de transferencias exige validar que las ofertas emitidas por los clubes cumplan condiciones de viabilidad deportiva, médica y financiera antes de formalizar un contrato y liquidar las comisiones de representación.

## 1.2. Descripción y objetivos del proyecto PL/SQL

El proyecto implementa la lógica de negocio en la base de datos mediante PL/SQL. El objetivo es automatizar las reglas de validación de las ofertas de fichaje, evitar inconsistencias operativas (ofertar por jugadores lesionados o no disponibles, superar el presupuesto del club o registrar ofertas duplicadas) y dejar un registro auditable de las operaciones. Para ello se desarrollan cuatro tipos de objetos que se usan entre sí: funciones, procedimientos, un package y triggers.

## 1.3. Datos procesados e información generada

El sistema procesa información estructurada en 13 entidades interconectadas:

- **Prospección y evaluación:** scout, evaluacion, posicion, jugador, prueba, participacion_prueba, historial_club y lesion.
- **Transaccionales y financieras:** club, oferta, contrato, comision y auditoria_oferta (tabla de apoyo creada para los triggers).

A partir de ellas se genera información como los jugadores que hoy pueden recibir ofertas, el presupuesto disponible de cada club, las ofertas rechazadas y las que siguen vigentes, y el registro de las operaciones realizadas sobre OFERTA. El script de carga deja 600 jugadores, 25 clubes y 200 ofertas.

## 1.4. Alcance

La solución abarca desde la evaluación de elegibilidad del jugador hasta el rechazo automático de las ofertas que dejan de cumplir las reglas y la auditoría de las operaciones a nivel de sentencia. Contribuye a la consistencia financiera y a reducir los errores humanos en la toma de decisiones de fichaje.

## 1.5. Tecnologías utilizadas

- **SGBD:** Oracle Database 19c / 21c.
- **Lenguaje:** PL/SQL.
- **Entorno de desarrollo:** Oracle SQL Developer.

## 1.6. Distribución de responsabilidades

| Integrante | Responsabilidad | Entregables a su cargo |
|---|---|---|
| Diego Jaramillo | Informe y contexto | Informe.pdf y Contexto.pdf |
| Italo Campodonico | Todo lo relacionado con el código | Requerimiento 1 a 4 (.SQL) y Script de Carga.SQL |
| Jose Salas | Presentación y corrección de código | Presentación.pptx y corrección del código fuente |

# 2. Desarrollo de procedimientos almacenados y funciones

## 2.1. Definiciones técnicas, características y diferencias

Una **función almacenada** es un subprograma PL/SQL guardado en la base de datos que calcula y devuelve un valor. Un **procedimiento almacenado** es un subprograma que ejecuta una tarea (por ejemplo, sentencias DML) y puede devolver resultados mediante parámetros de salida.

| Característica | Función almacenada (FUNCTION) | Procedimiento almacenado (PROCEDURE) |
|---|---|---|
| Retorno de valor | Debe retornar un único valor mediante la cláusula RETURN. | No usa RETURN para entregar un valor. Devuelve datos con parámetros OUT / IN OUT. |
| Invocación en SQL | Puede usarse en SELECT, WHERE, ORDER BY o HAVING (si no modifica el estado de la BD). | No se usa dentro de sentencias SQL; se ejecuta con EXEC o desde un bloque PL/SQL. |
| Propósito principal | Cálculos, validaciones de estado y transformación de datos. | Procesos de negocio, operaciones DML y orquestación de flujos. |

## 2.2. Ventajas y desventajas en entornos de base de datos

**Ventajas**

- **Rendimiento:** el código se compila y se almacena en la base de datos, lo que reduce el tráfico de red.
- **Seguridad:** permite conceder permisos de ejecución (EXECUTE) sin dar acceso directo a las tablas.
- **Reutilización:** la lógica de negocio queda centralizada y disponible para varios programas.

**Desventajas**

- Consumen CPU y memoria en el servidor de base de datos.
- Quedan acoplados al motor Oracle, lo que dificulta migrar a otro SGBD.

## 2.3. Propósito y análisis en el negocio

**A. Funciones de validación y cálculo (Requerimiento 2)**

**1. fn_lesion_activa(p_id_jugador IN NUMBER) RETURN NUMBER**

- **Propósito:** Determina si el jugador tiene una lesión grave en recuperación.
- **Lógica de negocio:** Busca en LESION un registro de gravedad GRAVE cuya fecha_lesion + dias_recuperacion_estimados sea mayor que SYSDATE. Retorna 1 si existe y 0 si no.

**2. fn_jugador_ofertable(p_id_jugador IN NUMBER) RETURN NUMBER**

- **Propósito:** Evalúa si un futbolista puede recibir ofertas.
- **Lógica de negocio:** Verifica que su estado sea DISPONIBLE o EN_NEGOCIACION y que no tenga lesión grave activa (usa fn_lesion_activa). Retorna 1 (ofertable), 0 (no ofertable) o -1 si el jugador no existe: la excepción NO_DATA_FOUND se captura y se retorna -1.

**3. fn_presupuesto_disponible(p_id_club IN NUMBER) RETURN NUMBER**

- **Propósito:** Calcula el saldo real que le queda a un club para fichar.
- **Lógica de negocio:** Resta del presupuesto del club la suma de sus ofertas en estado PENDIENTE. Puede ser negativo si el club ofertó más de lo que tiene. Si el club no existe captura NO_DATA_FOUND y retorna NULL.

**4. fn_jugadores_no_ofertables RETURN NUMBER**

- **Propósito:** Función sin parámetros que informa cuántos jugadores no pueden recibir ofertas hoy.
- **Lógica de negocio:** Cuenta los jugadores para los que fn_jugador_ofertable retorna 0. Con los datos cargados el resultado es 306. Se usa con SELECT fn_jugadores_no_ofertables FROM dual y dentro de sp_revisar_ofertas_pendientes.

**¿Por qué funciones?** Cada una devuelve un valor y no modifica datos; por eso pueden aplicarse a las 600 filas de JUGADOR (o a los 25 clubes) dentro de un SELECT y reutilizarse desde procedimientos, el package y los triggers.

**B. Procedimientos almacenados de procesamiento (Requerimiento 1)**

**1. sp_revisar_ofertas_club(p_id_club IN NUMBER, p_rechazadas OUT NUMBER)**

- **Propósito:** revisar las ofertas PENDIENTES de un club mediante un cursor explícito (c_ofertas), de la más antigua a la más nueva.
- **Lógica:** rechaza (estado RECHAZADA) las ofertas cuyo jugador ya no es ofertable (por lesión grave o por no estar disponible) o que el club ya no puede pagar. El presupuesto se va reservando por orden de llegada: cada oferta aceptable descuenta su monto del disponible. Entrega por el parámetro OUT la cantidad de ofertas rechazadas. Si el club no existe, captura NO_DATA_FOUND e informa el error.

**2. sp_revisar_ofertas_pendientes**

- **Propósito:** procedimiento sin parámetros que revisa de una vez todas las ofertas pendientes de la agencia.
- **Lógica:** recorre con un cursor (c_clubes) cada club que tiene ofertas pendientes, invoca a sp_revisar_ofertas_club y al final muestra el resumen: ofertas revisadas, rechazadas, vigentes y jugadores que hoy no se pueden ofrecer (fn_jugadores_no_ofertables).

**¿Por qué procedimientos?** Ejecutan UPDATE sobre muchas filas y coordinan un flujo; no se necesita un valor de retorno para usar dentro de una consulta SQL.

## 2.4. Evidencias de ejecución y manejo de NO_DATA_FOUND

Función dentro de una sentencia SQL aplicada a muchas filas (jugadores que figuran como disponibles pero tienen una lesión grave activa):

```
SELECT id_jugador,
       nombre || ' ' || apellido       AS jugador,
       estado,
       fn_lesion_activa(id_jugador)     AS lesion_activa,
       fn_jugador_ofertable(id_jugador) AS ofertable
  FROM jugador
 WHERE estado IN ('DISPONIBLE', 'EN_NEGOCIACION')
   AND fn_lesion_activa(id_jugador) = 1
 ORDER BY id_jugador;
```

Funciones desde un bloque anónimo, incluido el caso en que el dato no existe. Salida real:

```
Jugador 2 ofertable: 1
Jugador 4 ofertable: 0
Jugador 1 ofertable: 0
Jugador 9999 ofertable: -1
Club 999 disponible: NULL (no existe)
```

Cuando el jugador (9999) no existe, fn_jugador_ofertable retorna -1; cuando el club (999) no existe, fn_presupuesto_disponible retorna NULL. En ambos casos la consulta no se interrumpe.

Procedimiento con parámetros invocado desde un bloque anónimo (club 20) y con un club inexistente. Salida real:

```
Oferta 54 (Ezequiel Fuentes) rechazada: jugador no disponible
Oferta 155 (Facundo Araya) rechazada: jugador no disponible
Oferta 106 (Ezequiel Vidal) rechazada: jugador no disponible
```

```
  Oferta 35 (Cristian Vega) rechazada: jugador no disponible
  Oferta 88 (Ignacio Vega) rechazada: presupuesto insuficiente
Fenix Metropolitana (club 20): 5 ofertas rechazadas
Valor recibido por el OUT: 5
El club 999 no existe.
```

Procedimiento sin parámetros (revisión completa). Resumen de la salida real:

```
Ofertas revisadas: 75
Ofertas rechazadas: 48
Ofertas que siguen vigentes: 27
Jugadores que hoy no se pueden ofrecer: 306
```

# 3. Desarrollo de packages almacenados

## 3.1. Definición, especificación y cuerpo

Un package de PL/SQL agrupa bajo un solo nombre tipos, variables, constantes, funciones y procedimientos relacionados. Se compone de dos partes que se compilan por separado:

- **Especificación (PACKAGE):** define la interfaz pública. Todo lo declarado aquí puede ser usado por quien tenga permiso de ejecución.
- **Cuerpo (PACKAGE BODY):** contiene el código de las rutinas públicas y declara los elementos privados, que quedan ocultos al exterior.

## 3.2. Ventajas y desventajas

**Ventajas**

- **Encapsulamiento:** las constantes y funciones privadas no pueden ser invocadas ni alteradas desde fuera del package.
- **Carga en memoria:** al invocar por primera vez un elemento, el package completo se carga en memoria, lo que reduce accesos posteriores.
- **Estado de sesión:** las variables públicas conservan su valor durante la sesión.
- **Organización:** reúne en un solo objeto el código relacionado del proyecto.

**Desventajas**

- Si se modifica el cuerpo y la especificación cambia, se invalidan los objetos que dependen de ella.
- Cargar el package completo consume más memoria que un subprograma suelto, y el código es más extenso de mantener.

## 3.3. Estructura de pkg_ofertas

El package agrupa los objetos de los requerimientos 1 y 2 y los expone solo en lo necesario:

| Elemento | Tipo | Visibilidad | Parámetros | Para qué sirve |
|---|---|---|---|---|
| **g_ofertas_registradas** | Variable | Pública | — | Cuenta las ofertas registradas en la sesión. |
| **jugador_ofertable** | Función | Pública | p_id_jugador IN | Envuelve fn_jugador_ofertable; la usa el trigger de fila. |
| **presupuesto_disponible** | Función | Pública | p_id_club IN | Envuelve fn_presupuesto_disponible; sirve en un SELECT. |
| **registrar_oferta** | Procedimiento | Público | jugador, club, monto IN · id_oferta OUT | Aplica 4 reglas de negocio y registra la oferta como PENDIENTE. |

| Elemento | Tipo | Visibilidad | Parámetros | Para qué sirve |
|---|---|---|---|---|
| **revisar_ofertas** | Procedimiento | Público | — | Llama a sp_revisar_ofertas_pendientes (Req. 1). |
| **c_monto_minimo** | Constante | Privada | — | Monto mínimo de una oferta: 50000. Regla interna que nadie de afuera debe cambiar. |
| **oferta_duplicada** | Función | Privada | jugador, club IN | Evita dos ofertas pendientes del mismo club por el mismo jugador. |

**Por qué se oculta oferta_duplicada:** consulta la tabla OFERTA; si un trigger de fila sobre OFERTA la utilizara, Oracle produciría el error ORA-04091 (tabla mutante). Al dejarla privada solo la usa registrar_oferta, y el trigger de fila queda libre de ese riesgo. **Por qué se oculta c_monto_minimo:** es una regla interna que no debe modificarse desde fuera.

### Especificación del package (código real)

```
CREATE OR REPLACE PACKAGE pkg_ofertas IS

    -- Variable global: ofertas registradas en la sesion
    g_ofertas_registradas NUMBER := 0;

    -- Consultas que usan otras areas, SELECT y el trigger del Req. 4
    FUNCTION jugador_ofertable (
        p_id_jugador jugador.id_jugador%TYPE
    ) RETURN NUMBER;

    FUNCTION presupuesto_disponible (
        p_id_club club.id_club%TYPE
    ) RETURN NUMBER;

    -- Registra una oferta solo si cumple las 4 reglas de negocio
    PROCEDURE registrar_oferta (
        p_id_jugador IN  oferta.id_jugador%TYPE,
        p_id_club    IN  oferta.id_club%TYPE,
        p_monto      IN  oferta.monto_oferta%TYPE,
        p_id_oferta  OUT oferta.id_oferta%TYPE
    );

    -- Revision masiva de todas las ofertas pendientes
    PROCEDURE revisar_ofertas;

END pkg_ofertas;
/
```

## 3.4. Ejemplo de invocación y evidencias

```
DECLARE
    v_id oferta.id_oferta%TYPE;
BEGIN
    pkg_ofertas.registrar_oferta(2, 25, 500000, v_id);
    DBMS_OUTPUT.PUT_LINE('Id recibido por el OUT: ' || v_id);
END;
/
```

Salida real de las pruebas del package (oferta válida y cada regla de negocio que rechaza la oferta). Por diseño, registrar_oferta captura el error y muestra el mensaje en lugar de propagarlo:

```
Oferta 201 registrada: jugador 2, club 25, monto 500000
Id recibido por el OUT: 201
Oferta no registrada: ORA-20106: El club 25 ya tiene una oferta pendiente por el jugador
2.
Oferta no registrada: ORA-20103: El jugador 4 no esta disponible o tiene una lesion grave
en recuperacion.
Oferta no registrada: ORA-20103: El jugador 1 no esta disponible o tiene una lesion grave
en recuperacion.
Oferta no registrada: ORA-20105: El club 24 solo tiene 30125 disponible.
Oferta no registrada: ORA-20101: El monto minimo de una oferta es 50000.
Oferta no registrada: ORA-20102: El jugador 9999 no existe.
Oferta no registrada: ORA-20104: El club 999 no existe.
Ofertas registradas en la sesion: 1
```

Al intentar usar un elemento privado desde afuera (pkg_ofertas.oferta_duplicada) Oracle responde con el error de compilación PLS-00302: el componente 'OFERTA_DUPLICADA' se debe declarar, lo que confirma que está oculto.

# 4. Desarrollo de triggers de base de datos

## 4.1. Definición de momentos, eventos y niveles

Los triggers (disparadores) son bloques PL/SQL almacenados que se ejecutan automáticamente cuando ocurre un evento DML (INSERT, UPDATE o DELETE) sobre una tabla.

- **Momento (BEFORE / AFTER):** BEFORE ejecuta la lógica antes de grabar el cambio (ideal para validar); AFTER la ejecuta una vez aplicado el cambio (ideal para auditar).
- **Nivel de fila (FOR EACH ROW):** se dispara una vez por cada fila afectada y permite usar :NEW y :OLD.
- **Nivel de sentencia:** se dispara una sola vez por instrucción SQL, sin importar cuántas filas afecte; no tiene acceso a :NEW ni :OLD.

## 4.2. Ventajas y desventajas

- **Ventajas:** las reglas se cumplen siempre, aunque el cambio se haga con un INSERT o UPDATE directo, de forma centralizada y transparente para las aplicaciones.
- **Desventajas:** son más difíciles de depurar, pueden encadenarse de forma no deseada y pueden provocar el error de tabla mutante (ORA-04091) si consultan la tabla que los dispara.

## 4.3. Implementación de los triggers del sistema

| Trigger | Momento | Evento · tabla | Nivel | Qué hace y por qué ese momento |
|---|---|---|---|---|
| **trg_oferta_validar** | BEFORE | INSERT OR UPDATE OF monto_oferta, estado_oferta · OFERTA | Fila | Rechaza oferta nueva que no sea PENDIENTE (-20020), jugador no disponible (-20021), lesionado (-20022) o inexistente (-20025), monto sobre el presupuesto (-20023) y cambios a ofertas ya cerradas (-20024). Es BEFORE para impedir que el dato inválido se grabe. Usa :OLD y :NEW y la función pkg_ofertas.jugador_ofertable. |
| **trg_oferta_no_eliminar** | BEFORE | DELETE · OFERTA | Sentencia | Prohíbe borrar ofertas (-20030): deben quedar como RECHAZADA para conservar el historial. |
| **trg_oferta_auditoria** | AFTER | INSERT OR UPDATE · OFERTA | Sentencia | Inserta una fila en AUDITORIA_OFERTA (operación, usuario y fecha) por cada sentencia. Es AFTER para registrar solo operaciones que sí se realizaron. |

**Tabla mutante:** trg_oferta_validar no consulta OFERTA; solo lee CLUB y usa funciones que consultan JUGADOR y LESION. La regla de oferta duplicada, que sí exige consultar OFERTA, la cubre el package (función privada). **Tabla de apoyo:** AUDITORIA_OFERTA se crea en el Script de Carga (columnas id_auditoria, operacion, usuario y fecha).

### Trigger de sentencia que impide eliminar (código real)

```
CREATE OR REPLACE TRIGGER trg_oferta_no_eliminar
BEFORE DELETE ON oferta
BEGIN
    RAISE_APPLICATION_ERROR(-20030, 'Las ofertas no se eliminan; deben quedar como RECHAZADA para
mantener el historial.');
END trg_oferta_no_eliminar;
/
```

## 4.4. Demostración práctica y auditoría

Resultado real de las pruebas, ejecutadas después de las de los requerimientos 1 y 3:

| Prueba | Sentencia | Resultado |
|---|---|---|
| 1 | pkg_ofertas.registrar_oferta(449, 5, 400000, v_id) | Oferta 202 registrada; se dispara el trigger de sentencia. |
| 2 | INSERT directo, jugador 4 con lesión grave | ORA-20022: El jugador 4 tiene una lesion grave en recuperacion. |
| 3 | INSERT directo, jugador 1 no disponible | ORA-20021: El jugador 1 no esta disponible. |
| 4 | INSERT directo con estado ACEPTADA | ORA-20020: Toda oferta nueva debe registrarse como PENDIENTE. |
| 5 | INSERT directo de 900000 al club 20 | ORA-20023: El club 20 no tiene presupuesto suficiente (748453 < 900000). |
| 6 | UPDATE de la oferta 2 (ya ACEPTADA) | ORA-20024: La oferta 2 ya esta ACEPTADA y no puede modificarse. |
| 7 | UPDATE que rechaza las 3 pendientes del club 2 | 3 filas actualizadas: el trigger de fila se ejecuta 3 veces y el de sentencia 1 vez. |
| 8 | EXEC pkg_ofertas.revisar_ofertas (jugador 41 lesionado) | Oferta 121 rechazada por lesión grave; 26 revisadas, 1 rechazada, 25 vigentes. |
| 9 | DELETE FROM oferta WHERE id_oferta = 20 | ORA-20030: Las ofertas no se eliminan; deben quedar como RECHAZADA. |

Salida real de la prueba 7, que muestra la diferencia entre ambos niveles:

```
[trigger fila] Oferta 187: PENDIENTE -> RECHAZADA
[trigger fila] Oferta 189: PENDIENTE -> RECHAZADA
[trigger fila] Oferta 194: PENDIENTE -> RECHAZADA
[trigger sentencia] Operacion registrada en AUDITORIA_OFERTA
```

Las operaciones exitosas (pruebas 1, 7 y 8) dejaron un registro cada una en AUDITORIA_OFERTA; las que fallaron no dejan registro. Consulta de verificación:

```
SELECT * FROM auditoria_oferta ORDER BY id_auditoria;
```

# 5. Conclusión y recomendaciones

## 5.1. Resumen técnico y logros alcanzados

Se implementó en Oracle PL/SQL una solución integrada para validar y revisar las ofertas de la agencia de scouting. Se combinaron funciones de evaluación de estado, procedimientos de revisión masiva, un package que agrupa y encapsula las reglas, y triggers de fila y de sentencia que las hacen cumplir siempre. Con los datos cargados, la revisión masiva analizó 75 ofertas pendientes, rechazó 48 y dejó 27 vigentes.

## 5.2. Impacto de la solución PL/SQL en la agencia

- **Reglas de negocio centralizadas:** el trigger de fila valida cada INSERT o UPDATE sobre OFERTA, incluso si se hace sin pasar por el package, y las ofertas no pueden borrarse.
- **Automatización operativa:** un solo EXEC revisa todas las ofertas pendientes de los 25 clubes y rechaza las que ya no son viables, en lugar de revisarlas una por una.
- **Trazabilidad:** la tabla AUDITORIA_OFERTA registra qué operación se hizo, quién la hizo y cuándo.

## 5.3. Recomendaciones

- Auditar también los valores anteriores y nuevos de cada oferta usando :OLD y :NEW en un trigger de fila.
- Extender los controles y el package a las tablas CONTRATO y COMISION.

- Programar la revisión masiva de ofertas como un proceso periódico.

# 6. Anexos

## 6.1. Código completo

El código PL/SQL completo del proyecto se entrega como archivos independientes, según lo solicitado en la pauta: Requerimiento 1.sql, Requerimiento 2.sql, Requerimiento 3.sql, Requerimiento 4.sql y Script de Carga.sql, disponibles también en el repositorio de control de versiones en la rama evaluacion-parcial-2 (https://github.com/ExilRaizen/TallerBD_Clubfutbol).

## 6.2. Diagramas y modelos

El modelo de datos del proyecto se entrega como archivo independiente (Diagrama.jpg). Muestra las 13 entidades del modelo base (CLUB, JUGADOR, SCOUT, EVALUACION, PRUEBA, PARTICIPACION_PRUEBA, OFERTA, CONTRATO, LESION, HISTORIAL_CLUB, COMISION y POSICION, AUDITORIA_OFERTA), con sus atributos, claves y relaciones.

# 7. Bibliografía (APA)

El equipo utilizó el modelo de lenguaje Claude (Sonnet 5), de Anthropic, como apoyo puntual en: la revisión del código PL/SQL de los 4 requerimientos, aplicando algunas pequeñas correcciones en algunos requerimientos; la comprensión, en palabras breves, de qué contenidos correspondía redactar en cada sección del informe y la redacción del modelo APA. El diseño de la solución de negocio, las decisiones finales y la validación de cada script fueron responsabilidad del equipo.

Oracle. (s.f.). *Database PL/SQL language reference*. Oracle Help Center. https://docs.oracle.com/en/database/oracle/oracle-database/19/lnpls/

Gandolfi, C. (2026). *Guía de apoyo para el estudiante – Evaluación Parcial N.º 2, BDY1103 Taller de Base de Datos* [Documento de cátedra]. Duoc UC.

Anthropic. (2026). *Claude (Sonnet 4.5)* [Modelo de lenguaje de gran escala]. https://www.anthropic.com/claude